



Práctica 1. Clases y Objetos

Objetivos de aprendizaje:

1. Convenciones de codificación en Java	Aplicar adecuadamente las normas de estilo y convenciones de codificación de Java (nombres de variables, formato, comentarios, indentación, etc.), mostrando código claro y legible.
2. Comprensión de clases y objetos	Demostrar comprensión y uso correcto de la sintaxis y semántica de clases y objetos en Java, incluyendo la declaración, instanciación y uso de métodos/atributos.
3. Aplicación del encapsulamiento de la información	Saber implementar correctamente el encapsulamiento, definiendo atributos privados y métodos públicos según corresponda, evitando exponer detalles internos de las clases y utilizando correctamente los modificadores de acceso

Descripción

En esta primera práctica se inicia el programa que se irá ampliando a lo largo de las próximas sesiones. En las cinco secciones en que se compone esta práctica guiada se permite el uso de Inteligencia Artificial Generativa (IAG) solo para disponer de una retroalimentación final sobre el desarrollo del ejercicio.

1. Clase JuegoConVidas

Implementar una clase JuegoConVidas con las siguientes características:

- Atributos. Tiene como atributos privados:
 - ✓ **vidas**: Un entero que indica el número de vidas que le quedan al jugador en la partida actual (vidas restantes).
 - ✓ **vidasIniciales**: Un entero que guarde el número de vidas que inicialmente se le pasaron al constructor del objeto. Este atributo se utilizará para poder reiniciar el juego.
 - ✓ **record**: Un entero que guarde el récord. A diferencia de los anteriores (que son atributos de instancia) éste es un atributo de clase, por lo que será común a todos los juegos que se implementen. Inicialmente este atributo tendrá el valor 0.
- Métodos
 - ✓ Tiene como método el constructor que acepta un parámetro de tipo entero que indica el número de vidas iniciales con las que parte el jugador.
 - ✓ Tiene un método **muestraVidasIniciales** que visualiza por pantalla el número de vidas iniciales de la partida actual.

- ✓ Tiene un método **muestraVidasRestantes** que visualiza por pantalla el número de vidas que le quedan al jugador en la partida actual.
- ✓ Añadir un método **quitaVida** que disminuya en 1 el número de vidas del jugador y devuelva un *booleano* indicando si al jugador le quedan más vidas (*true*) o no (*false*). Además, en el caso de que le queden vidas debe mostrar por pantalla cuántas le quedan y si no mostrar el mensaje “Juego Terminado”.
- ✓ Añadir un método **reiniciaPartida** que asigne al atributo vidas el número de vidas iniciales con el que se instanció el objeto. Para ello utilizará el atributo *vidasIniciales*.
- ✓ Añadir un método **actualizaRecord** que compare el valor actual de *récord* con el número de vidas restantes.
 - Si el número de vidas restantes es igual al *récord*, mostrará un mensaje indicando que se ha alcanzado el *récord*.
 - Si el número de vidas restantes es mayor que el *récord*, actualizará el *récord* y mostrará un mensaje diciendo que éste se ha batido y cuál es su nuevo valor.
 - Si el número de vidas restantes es menor que el *récord*, no hará nada.

2. Clase principal.

Para probar la clase **JuegoConVidas** se procederá como sigue en el método **main** de la clase **principal**:

- ✓ Crea una instancia de la clase **JuegoConVidas** indicando que el número de vidas es 5 (primera instancia).
- ✓ Invoca el método **muestraVidasIniciales** a través del objeto creado.
- ✓ Invoca el método **quitaVida** a través de esta instancia.
- ✓ Crea otra instancia distinta de la clase **JuegoConVidas** indicando que el número de vidas es también de 5 (segunda instancia) e invoca el método **muestraVidasIniciales** a través de dicha instancia.
- ✓ Invoca el método **actualizaRecord** a través de esta segunda instancia de **JuegoConVidas**.
- ✓ Invoca el método **quitaVida** a través de esta segunda instancia.
- ✓ A continuación, y a través de la primera instancia, invoca el método **reiniciaPartida** y, a continuación, el método **muestraVidasRestantes**.
- ✓ Por último, invoca el método **actualizaRecord** a través de la primera instancia de **JuegoConVidas**.

Ejecutar el programa y comprender los mensajes mostrados. Explica brevemente por qué se obtienen dichas respuestas.

3. Evaluación y retroalimentación

3.1. Rúbrica de logro de resultados de aprendizaje

Criterio	Consigue el resultado	No consigue el resultado
1. Convenciones de codificación	Aplica correctamente las normas de estilo Java y justifica sus decisiones mostrando comprensión de su importancia para la legibilidad y el mantenimiento del código.	No aplica correctamente las normas de estilo o no demuestra comprender su finalidad.
2. Comprensión de clases y objetos	Declara, instancia y utiliza clases/objetos correctamente, y explica con claridad el funcionamiento y la diferencia entre atributos de instancia y de clase (estáticos).	Comete errores conceptuales o técnicos al usar clases/objetos, o no comprende las diferencias entre atributos de instancia y de clase.
3. Aplicación del encapsulamiento de la información	Implementa adecuadamente el encapsulamiento y razona sobre las ventajas de ocultar información y utilizar modificadores de acceso apropiados.	No implementa correctamente el encapsulamiento o no comprende su propósito y ventajas.

3.2. Preguntas de autoevaluación.

Responde a las siguientes preguntas, justificando tus respuestas con fragmentos de código o ejemplos concretos desarrollados en la práctica:

1. ¿Qué convenciones de codificación de Java has aplicado en la implementación de la clase JuegoConVidas? Menciona ejemplos concretos relacionados con el nombrado de variables y métodos, la indentación, los comentarios y el formato del código, y explica brevemente cómo estas convenciones contribuyen a mejorar la legibilidad y el mantenimiento del programa.
2. Explica cómo se declara, instancia y utiliza un objeto de la clase JuegoConVidas en el método main. ¿Cuál es la diferencia entre los atributos de instancia y los atributos de clase en tu implementación?
3. ¿Por qué los atributos de la clase JuegoConVidas se han definido como privados y los métodos como públicos? ¿Qué ventajas aporta este diseño en cuanto a la ocultación de la información?
4. Describe el funcionamiento de los métodos quitaVida y actualizaRecord. Explica qué ocurre internamente cuando se ejecutan y cómo afectan al estado de cada objeto. ¿Por qué estos métodos son métodos de instancia y no métodos estáticos?
5. Una vez ejecutado el programa, interpreta los mensajes obtenidos en pantalla y justifica por qué aparecen esos resultados. En tu explicación, ten en cuenta que el atributo record es común a todas las instancias. ¿Qué nos indica esto acerca del modo en que funcionan los atributos estáticos en Java?

3.3. Obteniendo retroalimentación de la IAG

A continuación, deberás solicitar a la IAG con la que trabajes una primera evaluación (retroalimentación) sobre la corrección de tu código y las respuestas que has dado a las preguntas planteadas. Para ello elabora un prompt lo más detallado posible que permita a la IAG realizar una evaluación de la corrección tanto del código como de las respuestas proporcionadas. Es importante que en tu prompt incluyas:

- El enunciado de la práctica
- Los objetivos de aprendizaje
- La rúbrica de logro de resultados de aprendizaje.
- El código desarrollado
- Las preguntas de autoevaluación junto con tus respuestas

La valoración recibida **no debe ser una nota numérica**, sino un análisis en el que se indiquen:

- Qué aspectos están bien resueltos.
- Qué errores o carencias se detectan.
- Qué aspectos son mejorables.
- Si se han alcanzado los objetivos y resultados de aprendizaje previstos, tanto en el código como en las respuestas teóricas.

Es fundamental que tu prompt recoja con claridad todos los aspectos mencionados, de modo que la IAG oriente su análisis exactamente a lo que se espera en la evaluación. Si la respuesta de la IAG no es suficientemente clara, ajustada o útil, mejora tu prompt añadiendo explicaciones adicionales, ejemplos concretos o aclaraciones sobre lo que necesitas.

Recuerda: **la calidad de la retroalimentación recibida depende de la precisión y el contexto que incluyas en tu consulta**. Compara siempre la respuesta de la IAG con los materiales de clase u otras fuentes fiables, o consulta con el/la profesor/a en caso de dudas persistentes. Esto te ayudará a desarrollar un uso crítico y responsable de la IAG.

4. Informe que debe subirse al campus virtual de la asignatura

Una vez obtengas una respuesta satisfactoria, sube a la tarea correspondiente en Moodle un informe en PDF, **utilizando para ello la plantilla que está disponible en el campus virtual**. Este informe incluye:

- Nombre y apellidos
- La IAG utilizada, su versión (si procede) y la fecha.
- El desarrollo de la práctica: El código de la práctica y las respuestas a las preguntas que se formulan en esta práctica.
- Interacción con la IAG: Transcribe el diálogo íntegro y ordenado (tus prompts y las respuestas de la IAG).
- Una valoración personal sobre la utilidad y limitaciones de la retroalimentación recibida de la IAG, indicando qué mejoras podrás incorporar gracias a ella (si la hubiera) y reflexionando brevemente sobre su impacto en tu aprendizaje.
- La declaración de autoría y uso de IAG, conforme a lo indicado en el siguiente apartado.

Puedes aprovechar las observaciones de la IAG para mejorar el código o tus respuestas en la práctica siguiente.

Nota: Es obligatorio entregar el informe en PDF utilizando la plantilla que se facilita en el campus virtual de la asignatura y completar correctamente todos sus apartados; cualquier entrega que no se ajuste a esta plantilla —por ejemplo, subir solo el código o un PDF que no se corresponda con la plantilla— se considerará no válida y conllevará el suspenso de toda la práctica guiada (las cinco entregas).

5. Uso de la IAG, autoría y responsabilidad académica

El uso de herramientas de IAG está permitido en esta práctica únicamente para obtener retroalimentación y sugerencias de mejora sobre el trabajo realizado (por ejemplo, evaluación de código, propuestas de corrección o ayuda en la redacción final). No se permite la elaboración inicial de código, respuestas o informes previos mediante IAG. Todo el desarrollo y resolución de la práctica debe ser realizado personalmente por el estudiante.

Para asegurar la transparencia, la responsabilidad y el uso adecuado de la IAG, la plantilla para la elaboración del informe incluye un apartado final de “Declaración de autoría y uso de IAG”. Mediante esta declaración, el estudiante manifiesta que el trabajo presentado ha sido realizado por él/ella, que asume la responsabilidad sobre su contenido y sobre las decisiones adoptadas durante su elaboración, y que la información proporcionada en el informe acerca del uso de herramientas de Inteligencia Artificial Generativa es completa y veraz.

La entrega del informe supone la aceptación expresa de esta declaración y la asunción de responsabilidad sobre el trabajo presentado.

Recuerda: El uso responsable de la IAG contribuye al aprendizaje autónomo y a la mejora continua, pero no puede sustituir al desarrollo personal de competencias técnicas y críticas. El falseamiento de la autoría será considerado una mala praxis académica.